

Projet Nichoir



Travail Réalisé par Jourdan Maxim et Monthe Modeste

Année 2025-2026 Passerelle Électronique M13P

Introduction	4
Cahier des Charges	4
Objectifs généraux	4
Exigences fonctionnelles	5
Schéma	5
Serveur MQTT Raspberry Pi	6
Pourquoi le protocole MQTT	6
Mise en place du serveur MQTT sur Raspberry Pi	6
Organisation des échanges MQTT	6
Transmission des images par fragmentation	6
Application Flask	7
Architecture de l'application	7
Fonctionnement général	7
Interface Web	8
Extensibilité du système	8
M5 STACK Esp32 Camera	8
Accès Wi-Fi et point d'accès de configuration	9
Détection de mouvement et déclenchement des captures	9
Transmission des données vers le serveur	10
Gestion de l'énergie et modes de veille	10
Capteur Infrarouge PIR	10
Principe de fonctionnement	10
Description des connexions du module PIR	11
GND (Ground)	12
3.3 V (Alimentation)	12
Photoresist (Entrée photorésistance)	12
Potentiomètre de sensibilité (Pot Sensi)	12
Potentiomètre de temporisation (Pot Time)	12
OUT (Sortie numérique)	13
Board 5v PIR	13
Schéma	14
Alimentation et conversion de tension	14
Conditionnement du capteur PIR	15
Intégration de la LED d'éclairage	15
Interface avec le module ESP32 TimerCAM	16
Intérêt de la board dédiée	16

Dessin 3D	16
Étapes de conception et apprentissage	17
Logiciel utilisé et méthodologie	18
Fonctionnalité et sécurité thermique	19
Résultat	19
Tests	19
Test du capteur PIR	20
Transmission des images via MQTT	20
Autonomie énergétique	20
Limites	20
Réveil depuis le mode veille du PIR	20
Transmission réseau dépendante de la stabilité Wi-Fi	21
Autonomie réelle vs autonomie théorique	21
Exposition aux éléments	21
Conclusion	21
Perspectives	22
Intégration d'un réseau LoRa Mesh	22
Alimentation solaire	22
Optimisation du mode veille	22
Extension des fonctionnalités	23
Scalabilité et modularité	23

Introduction

Le développement des villes intelligentes SmartCities s'accompagne d'un intérêt croissant pour le suivi de l'environnement urbain et de la biodiversité. La faune, et en particulier les oiseaux, constitue un indicateur pertinent de la qualité écologique des milieux urbains. Le suivi de leur comportement et de leur présence permet d'apporter des informations utiles tant pour la recherche scientifique que pour la sensibilisation du public à la préservation de l'environnement.

Dans ce contexte, le projet de nichoir connecté, réalisé dans le cadre du cours de passerelle Smart Cities, s'inscrit dans une démarche de conception d'un système IoT autonome, communicant et à faible consommation énergétique. L'objectif principal est de concevoir un nichoir intelligent capable de détecter la présence d'oiseaux, de capturer des images de manière non-intrusive et de transmettre ces données vers une infrastructure distante pour leur stockage et leur visualisation.

Le projet vise à proposer une solution économiquement accessible, avec un coût de fabrication inférieur aux solutions existantes du marché, tout en garantissant une autonomie énergétique prolongée grâce à l'utilisation d'une batterie et à une gestion optimisée de la consommation. L'ensemble du système repose sur un microcontrôleur embarqué intégrant une caméra, un capteur de mouvement infrarouge passif (PIR), ainsi qu'une communication Wi-Fi vers un serveur distant basé sur un Raspberry Pi.

Ce rapport présente l'ensemble des étapes de conception, de réalisation et de validation du nichoir connecté. Il aborde successivement l'architecture globale du système, les choix technologiques, la gestion de l'énergie, l'infrastructure de communication et de traitement des données, ainsi que les tests réalisés. Enfin, les limites du système actuel et les perspectives d'amélioration sont discutées.

Cahier des Charges

Objectifs généraux

Le système doit permettre :

La surveillance de l'activité à l'intérieur d'un nichoir à l'aide d'un système de détection de mouvement.

La capture d'images lors de la détection d'une présence.

La transmission des données vers un serveur distant pour stockage et visualisation.

Une autonomie énergétique prolongée, compatible avec une utilisation sur plusieurs mois sans intervention humaine.

Un coût global réduit, adapté à une solution pédagogique et déployable à grande échelle.

Exigences fonctionnelles

Système embarqué:

Utilisation d'un module M5Stack ESP32 Camera (TimerCam).

Détection de mouvement à l'aide d'un capteur infrarouge passif (PIR).

Capture d'images déclenchée par le capteur PIR.

Éclairage infrarouge (LED IR) lors de la prise de vue afin de limiter l'impact sur les oiseaux.

Gestion des modes veille (sleep) afin de réduire la consommation énergétique.

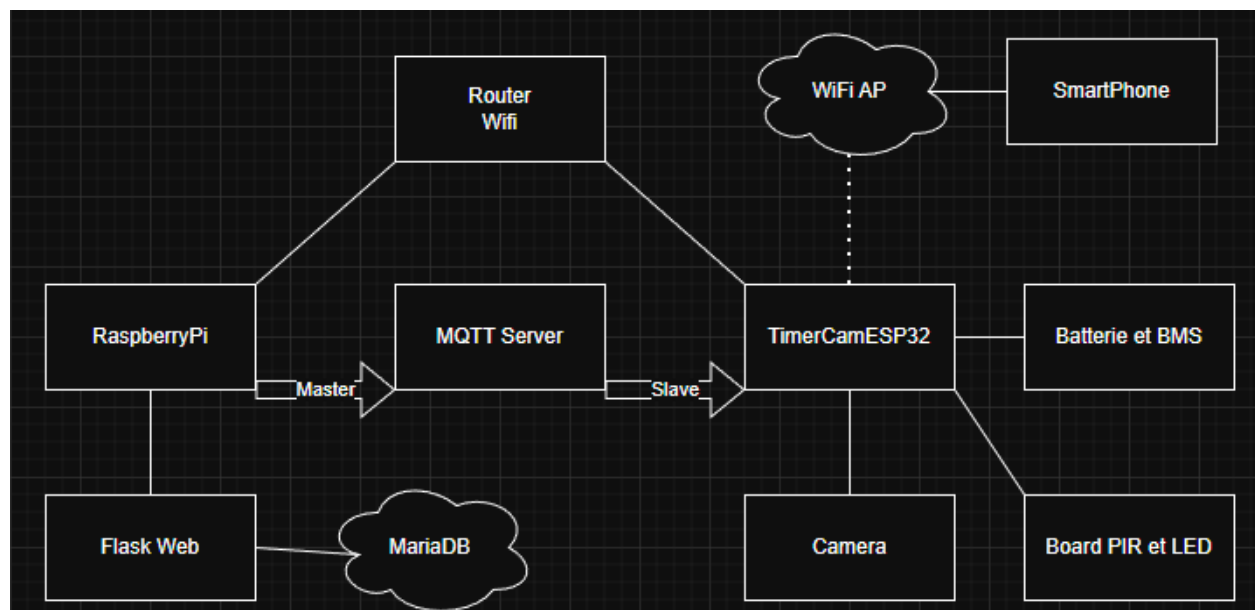
Connectivité:

Mise en place d'un point d'accès Wi-Fi permettant la configuration initiale du réseau.

Connexion automatique au réseau Wi-Fi configuré.

Transmission des images et des données via le protocole MQTT.

Schéma



Serveur MQTT Raspberry Pi

Pourquoi le protocole MQTT

Le protocole MQTT (Message Queuing Telemetry Transport) a été retenu pour sa légèreté et sa faible consommation réseau, adaptées aux systèmes IoT alimentés sur batterie. Son modèle publish/subscribe permet une communication asynchrone efficace, contrairement à HTTP, plus coûteux en ressources. MQTT facilite également l'évolutivité du système, plusieurs services pouvant exploiter les mêmes données sans modifier le module embarqué.

Mise en place du serveur MQTT sur Raspberry Pi

Un Raspberry Pi est utilisé comme serveur central hébergeant le broker MQTT Mosquitto. Il assure la réception des images et des métadonnées envoyées par l'ESP32, ainsi que leur redistribution vers les services applicatifs. La configuration du broker permet la réception de messages volumineux nécessaires à la transmission des images.

Organisation des échanges MQTT

Les échanges sont structurés à l'aide de topics MQTT distincts, séparant les images, les métadonnées (horodatage, niveau de batterie) et les informations de diagnostic. Cette organisation facilite le traitement et l'exploitation des données par le serveur applicatif.

Transmission des images par fragmentation

Les images étant trop volumineuses pour un envoi unique, elles sont fragmentées en plusieurs blocs transmis successivement via MQTT. Un message de fin permet la reconstitution de l'image côté serveur, garantissant une transmission fiable tout en respectant les contraintes du protocole.

Dans une version future du système, des mécanismes de sécurisation des échanges MQTT (authentification, chiffrement TLS) pourraient être intégrés afin d'améliorer la protection des données transmises.

L'ensemble des scripts et commandes est disponible sur le dépôt GitHub du projet.

Application Flask

Afin de permettre la visualisation des données collectées par le nichoir connecté, une application Web a été développée à l'aide du framework Flask, exécutée sur le Raspberry Pi. Flask a été choisi pour sa légèreté, sa simplicité d'intégration avec Python et sa pertinence dans des projets IoT de petite à moyenne échelle.

L'application Flask joue le rôle d'interface entre la base de données et l'utilisateur final. Elle permet de consulter les images capturées par le module ESP32 TimerCAM ainsi que les métadonnées associées, telles que la date de capture et le niveau de batterie du système embarqué.

Architecture de l'application

L'application Web est intégrée dans un script unifié exécuté sur le Raspberry Pi, combinant :

- un listener MQTT chargé de recevoir les images et les métadonnées,
- une base de données MariaDB assurant le stockage persistant,
- un serveur Web Flask permettant l'affichage et l'accès aux données.

Cette architecture permet un fonctionnement continu et autonome du système, tout en limitant le nombre de services à gérer sur le Raspberry Pi.

Fonctionnement général

Lorsqu'une image est reçue via le protocole MQTT, celle-ci est sauvegardée sur le système de fichiers du Raspberry Pi. Le chemin de l'image ainsi que les informations associées sont ensuite enregistrés dans la base de données. L'application Flask interroge cette base afin de récupérer l'ensemble des captures disponibles et de les présenter à l'utilisateur via une interface Web.

La page principale de l'application affiche une galerie d'images, organisée par ordre chronologique inverse. Chaque capture est accompagnée de métadonnées permettant d'évaluer l'état du système au moment de la prise de vue, notamment le niveau de batterie.

Interface Web

L'interface Web a été conçue afin d'être claire, responsive et facilement exploitable sur différents supports. Elle propose :

- une galerie d'images issue des captures du nichoir,
- des indicateurs synthétiques tels que le nombre total de captures et le niveau de batterie moyen,
- une mise en forme facilitant l'analyse visuelle des données collectées.

Un mécanisme de rafraîchissement périodique permet d'actualiser automatiquement l'affichage, assurant ainsi la consultation des nouvelles images sans intervention manuelle.

Extensibilité du système

En complément de l'interface graphique, l'application Flask expose des routes dédiées permettant d'accéder aux données sous forme structurée. Cette approche rend le système évolutif et compatible avec de futures extensions, telles que l'intégration d'outils d'analyse, de supervision ou de visualisation avancée.

Ainsi, l'application Flask constitue un élément central du projet, transformant le Raspberry Pi en un serveur Web complet, capable de valoriser les données issues du nichoir connecté de manière efficace et accessible.

M5 STACK Esp32 Camera

Le système embarqué du nichoir repose sur un module M5Stack ESP32 TimerCAM, intégrant un microcontrôleur ESP32, une caméra et une gestion native de l'alimentation sur batterie. Ce module a été choisi pour sa compacité, sa faible consommation énergétique et sa capacité à combiner acquisition d'images et connectivité Wi-Fi au sein d'un même dispositif.

L'ESP32 TimerCAM constitue le cœur du système IoT. Il assure la détection d'événements à l'intérieur du nichoir, la capture des images, ainsi que leur transmission vers le serveur distant hébergé sur Raspberry Pi.

Accès Wi-Fi et point d'accès de configuration

Afin de faciliter le déploiement du nichoir dans différents environnements, un mécanisme de configuration réseau dynamique a été mis en place. Lors du premier démarrage, ou en l'absence de réseau connu, le module ESP32 crée automatiquement un point d'accès Wi-Fi dédié.

Ce point d'accès permet à l'utilisateur de se connecter au module et de configurer les identifiants du réseau Wi-Fi local sans avoir à modifier le code embarqué. Une fois les paramètres enregistrés, l'ESP32 se connecte automatiquement au réseau configuré lors des démarrages suivants.

Cette approche améliore la flexibilité du système et rend son installation accessible, même sans compétences avancées en programmation embarquée.

Détection de mouvement et déclenchement des captures

La détection de présence à l'intérieur du nichoir est réalisée à l'aide d'un capteur infrarouge passif (PIR). Lorsqu'un mouvement est détecté, le capteur déclenche une séquence d'actions automatisées.

Dans un premier temps, une LED est activée afin d'éclairer brièvement la scène, garantissant une qualité d'image suffisante même dans des conditions de faible luminosité. La caméra capture ensuite une image de l'intérieur du nichoir.

Une fois la capture effectuée, l'image est immédiatement préparée pour être transmise vers le serveur distant. Ce fonctionnement événementiel permet de limiter les captures inutiles et de réduire la consommation énergétique globale du système.

En complément, des captures périodiques sont également réalisées à intervalles réguliers afin d'assurer un suivi continu de l'activité, même en l'absence de détection de mouvement.

Transmission des données vers le serveur

Après chaque capture, l'image ainsi que les métadonnées associées (horodatage, niveau de batterie, identifiant du dispositif) sont envoyées via le réseau Wi-Fi en utilisant le protocole MQTT.

En raison des contraintes mémoire du microcontrôleur et de la taille des images, celles-ci sont fragmentées en plusieurs blocs avant d'être transmises. Le serveur reconstitue ensuite l'image complète à la réception de l'ensemble des fragments. Cette méthode garantit une transmission fiable tout en respectant les limitations matérielles de l'ESP32.

Gestion de l'énergie et modes de veille

L'autonomie énergétique constitue un enjeu majeur du projet. Le module ESP32 TimerCAM fonctionne sur batterie et intègre plusieurs mécanismes visant à réduire sa consommation.

Des phases de mise en veille sont utilisées entre les captures afin de limiter l'activité du microcontrôleur. Des tests ont été réalisés avec différents modes de veille, notamment le deep sleep, afin d'évaluer leur compatibilité avec la détection de mouvement via le capteur PIR.

Ces expérimentations ont mis en évidence certaines limitations, notamment la difficulté de réveiller de manière fiable le système depuis un état de veille profonde à l'aide du capteur PIR. Ces contraintes ont conduit à des compromis entre autonomie énergétique et fiabilité de la détection, qui sont discutés plus en détail dans la section dédiée aux limites du système.

Capteur Infrarouge PIR

Principe de fonctionnement

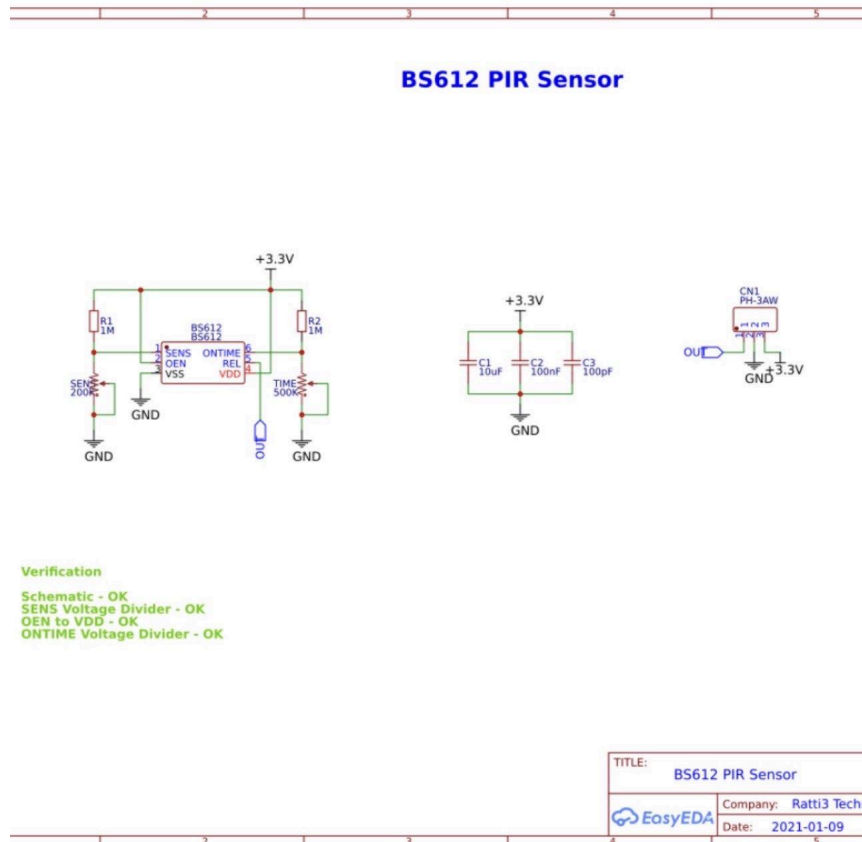
Le capteur infrarouge passif (PIR – Passive Infrared Sensor) est utilisé pour détecter la présence ou le mouvement d'un corps chaud dans son champ de vision. Il fonctionne en mesurant les variations du rayonnement infrarouge émis naturellement par les objets, et en particulier par les êtres vivants.

Le capteur PIR ne génère pas de rayonnement ; il se contente de détecter les variations d'énergie infrarouge reçues. Lorsqu'un oiseau entre dans le nichoir ou se déplace à proximité du capteur, la variation thermique est détectée et convertie en un signal électrique exploitable par le microcontrôleur.

Ce type de capteur présente plusieurs avantages pour un système autonome :

- faible consommation énergétique
- fonctionnement simple
- absence d'émission active, donc non-intrusif pour les animaux

Ces caractéristiques en font un choix particulièrement adapté au projet de nichoir connecté.



Description des connexions du module PIR

Le module PIR utilisé dans ce projet dispose de six connexions, permettant d'alimenter le capteur, de régler son comportement et de récupérer le signal de détection.

GND (Ground)

La broche GND correspond à la masse du circuit. Elle est reliée à la masse commune du système afin d'assurer une référence électrique stable entre le capteur PIR et le microcontrôleur ESP32.

3.3 V (Alimentation)

La broche 3.3 V permet d'alimenter le module PIR. Dans ce projet, le capteur est alimenté par la tension fournie par le système embarqué. L'utilisation d'une tension compatible avec l'ESP32 évite l'ajout de composants de conversion supplémentaires.

Photoresist (Entrée photorésistance)

La broche photoresist est associée à une photorésistance intégrée au module. Elle permet d'adapter le comportement du capteur en fonction de la luminosité ambiante.

Cette entrée peut être utilisée pour inhiber la détection lorsque la lumière est suffisante, ou au contraire pour privilégier les détections en conditions de faible luminosité. Dans le cadre du projet, cette fonctionnalité a été identifiée comme une possibilité d'amélioration, mais n'a pas été exploitée pleinement.

Potentiomètre de sensibilité (Pot Sensi)

Le potentiomètre de sensibilité permet de régler la distance ou l'amplitude de détection du capteur PIR. Un réglage élevé augmente la portée du capteur, tandis qu'un réglage plus faible limite la zone de détection.

Ce paramètre est important afin d'éviter les déclenchements intempestifs causés par des mouvements extérieurs au nichoir, tels que le vent ou des variations thermiques non pertinentes.

Potentiomètre de temporisation (Pot Time)

Le potentiomètre de temporisation définit la durée pendant laquelle la sortie du capteur reste à l'état actif après une détection. Il permet de contrôler le temps durant lequel le signal de sortie reste à l'état "haut".

Ce réglage est utilisé pour adapter la durée de déclenchement à l'application, par exemple afin de laisser suffisamment de temps au système pour capturer et transmettre une image avant un retour à l'état de veille.

OUT (Sortie numérique)

La broche OUT est la sortie logique du capteur PIR. Elle passe à l'état "haut" lorsqu'un mouvement est détecté et revient à l'état "bas" après la durée définie par le potentiomètre de temporisation.

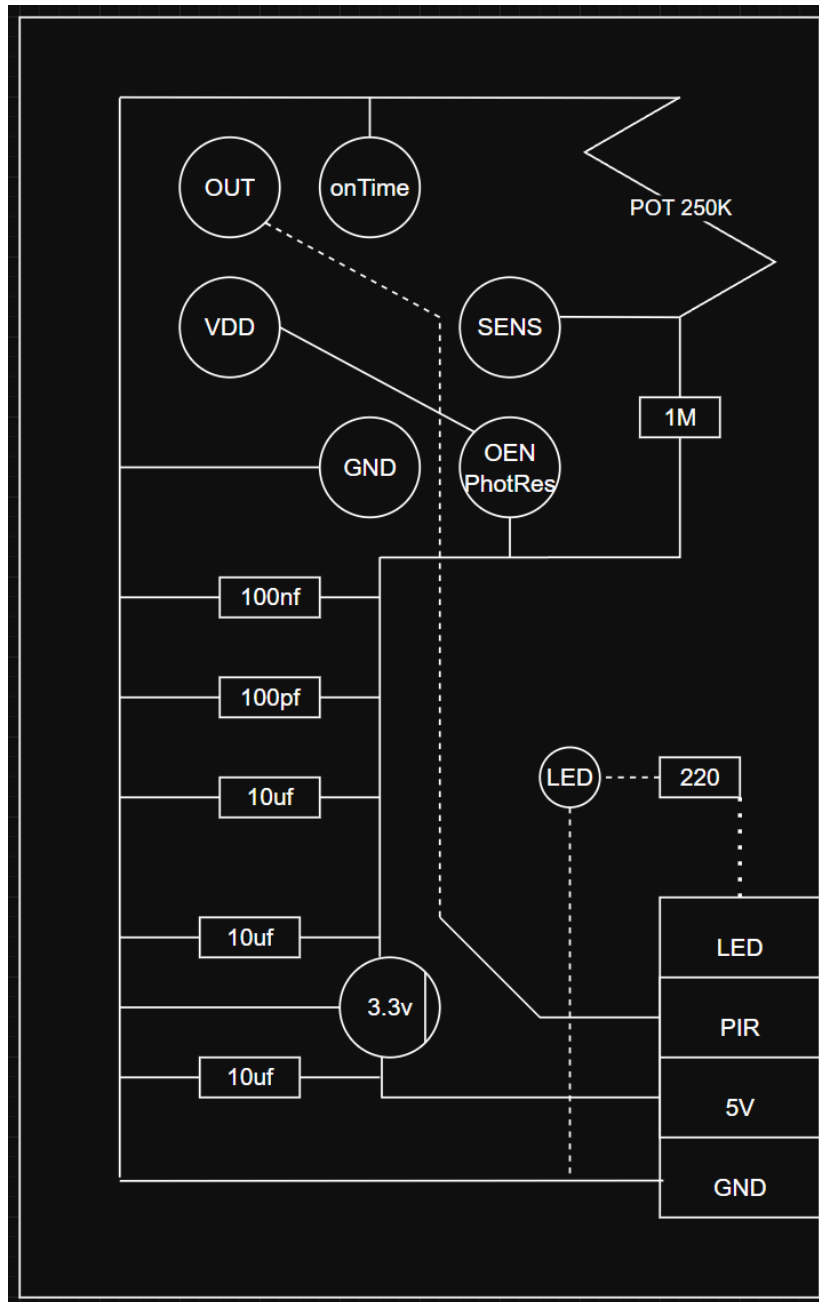
Cette sortie est directement reliée à une entrée numérique de l'ESP32, permettant de déclencher les actions logicielles : activation de la LED, capture d'image et transmission des données.

Board 5v PIR

Afin d'adapter le capteur PIR et la LED d'éclairage aux contraintes d'alimentation du système embarqué, une carte électronique dédiée a été conçue et réalisée. Cette board intermédiaire permet de centraliser l'alimentation, le conditionnement des signaux et l'interface avec le module ESP32 TimerCAM.

L'objectif principal de cette carte est de fournir une alimentation stable au capteur PIR en 3,3 V, tout en exploitant une alimentation principale en 5 V, et de simplifier les connexions vers le système embarqué.

Schéma



Alimentation et conversion de tension

Le système est alimenté en 5 V, notamment pour la LED d'éclairage. Cependant, le capteur PIR fonctionne de manière optimale sous 3,3 V, tension compatible avec les entrées de l'ESP32.

Un régulateur de tension $5\text{ V} \rightarrow 3,3\text{ V}$ est intégré sur la board afin d'assurer une alimentation stable du capteur PIR. Ce régulateur est accompagné de deux condensateurs de découplage de $10\text{ }\mu\text{F}$, placés respectivement à l'entrée et à la sortie du régulateur, afin de :

- lisser les variations de tension,
- améliorer la stabilité du régulateur,
- réduire le bruit électrique.

Conditionnement du capteur PIR

Le capteur PIR utilisé nécessite un ajustement de sa sensibilité afin d'être adapté à l'environnement restreint du nichoir. Pour cela, un réseau de composants passifs a été ajouté sur la board.

Un potentiomètre de $200\text{ k}\Omega$ et une résistance de $1\text{ M}\Omega$ sont utilisés pour former un pont diviseur, permettant d'ajuster finement la sensibilité du capteur. Ce réglage est essentiel pour éviter les déclenchements intempestifs causés par des variations thermiques non pertinentes.

Afin de garantir une alimentation propre et stable du capteur PIR, trois condensateurs ont été ajoutés sur la ligne $3,3\text{ V}$:

$10\text{ }\mu\text{F}$ pour le filtrage basse fréquence,
 100 nF pour le découplage classique,
 100 pF pour la suppression des parasites haute fréquence.

Cette combinaison permet d'assurer un fonctionnement fiable du capteur, même dans un environnement soumis à des perturbations électriques.

Intégration de la LED d'éclairage

La board intègre également la gestion de la LED utilisée pour l'éclairage lors de la capture d'image. La LED est alimentée directement en 5 V et associée à une résistance de limitation de courant, dimensionnée afin de protéger la LED et d'assurer une luminosité suffisante sans surconsommation.

L'activation de la LED est commandée par le microcontrôleur, permettant de synchroniser précisément l'éclairage avec la prise de vue, tout en minimisant la durée d'activation afin de réduire l'impact énergétique et lumineux sur les oiseaux.

Interface avec le module ESP32 TimerCAM

La board a été conçue pour communiquer simplement avec le module ESP32 TimerCAM à l'aide de quatre fils :

GND : masse commune du système,
5 V : alimentation principale,
LED_IN : commande de la LED,
PIR_OUT : signal de détection du capteur PIR.

Ces signaux sont reliés au connecteur du module ESP32 utilisant l'interface I²C (SCL / SDA) disponible sur le TimerCAM. Cette approche permet une intégration propre et compacte, sans modification matérielle du module principal.

Intérêt de la board dédiée

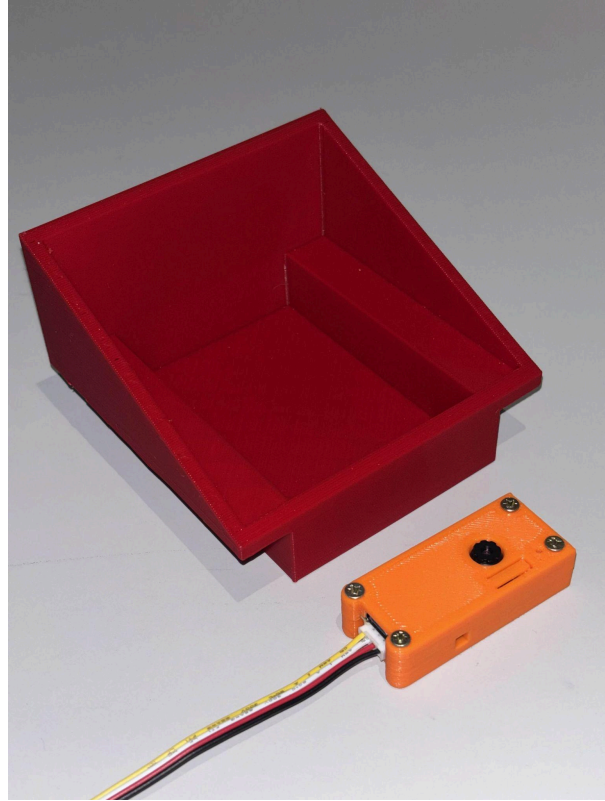
La réalisation de cette board 5 V PIR présente plusieurs avantages :

- séparation claire entre alimentation, détection et commande,
- amélioration de la fiabilité du capteur PIR,
- réduction du câblage direct vers l'ESP32,
- facilité de maintenance et d'évolution du système.

Cette approche modulaire s'inscrit pleinement dans une démarche de conception adaptée à un projet IoT autonome destiné à fonctionner sur de longues périodes sans intervention.

Dessin 3D

La conception du boîtier du nichoir a été réalisée afin de protéger le module ESP32 TimerCAM, le capteur PIR, la LED et la batterie, tout en s'intégrant facilement dans le nichoir et en assurant une bonne aération. L'objectif principal était de créer un boîtier pratique, spacieux, fonctionnel et compatible avec l'impression 3D.

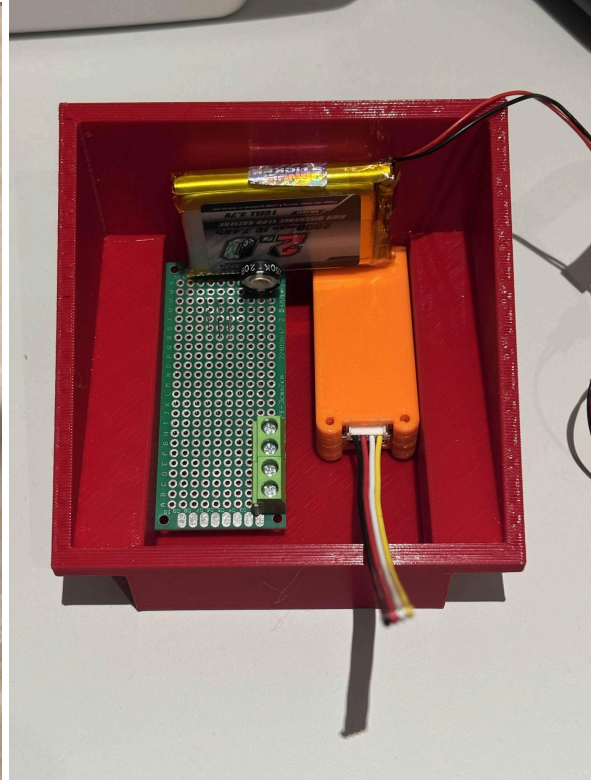


Étapes de conception et apprentissage

La conception a débuté par la création d'un boîtier hyper compact, destiné uniquement au module ESP32 TimerCAM. Cinq versions d'impression ont été réalisées pour tester la taille, l'ergonomie et la protection du module pendant le développement. Ces essais ont permis d'apprendre plusieurs notions importantes de l'impression 3D, notamment :

- comment ajouter des murs, rainures et plots pour maintenir les composants correctement en place,
- comment éviter que des objets ne se touchent ou ne se chevauchent dans l'espace 3D,
- comment concevoir des ouvertures pour les fils et les boutons tout en conservant l'étanchéité relative et la protection contre la poussière.

Cette expérience initiale a été très utile pour ensuite créer un boîtier plus grand et complet, capable d'accueillir l'ensemble du système (ESP32, PIR, LED, batterie).



Logiciel utilisé et méthodologie

Le logiciel Fusion 360 a été utilisé pour concevoir le boîtier. La méthodologie suivie a été la suivante :

Sketch 2D : création de profils en deux dimensions correspondant aux faces du boîtier ou aux ouvertures pour les composants.

Extrusion en bloc 3D : transformation des sketches 2D en volumes solides, donnant naissance aux différentes parties du boîtier.

Fusion et assemblage des blocs : les différents volumes ont été combinés pour former l'architecture finale, en veillant à ce que chaque composant ait son espace dédié.

Ajout de détails fonctionnels :

- trous pour le passage des fils de batterie,
- encoches pour les boutons,
- ouvertures pour le capteur PIR et la LED,
- logement pour la caméra.

Cette approche modulaire a permis de gérer efficacement l'espace et de garantir la facilité de montage et de maintenance.

Fonctionnalité et sécurité thermique

Le boîtier final offre une protection complète tout en permettant une aération suffisante : les trous pour les fils et les boutons contribuent à l'évacuation de la chaleur, et la température à l'intérieur du boîtier reste dans des limites sûres pour le fonctionnement du module ESP32.

Malgré ces ouvertures, le boîtier reste étanche une fois en position dans le nichoir, protégeant les composants de l'humidité et de la poussière. Le câblage à l'intérieur est organisé de manière compacte et sécurisée, sans risque de court-circuit ou de tension excessive sur les fils.

Résultat

Le boîtier 3D final :

- protège tous les composants,
- laisse suffisamment de place pour la batterie,
- facilite le montage et le démontage,
- assure une aération adaptée tout en restant étanche,
- est esthétique et adapté à l'environnement du nichoir.

Cette étape a constitué un apprentissage concret de la modélisation 3D et de l'impression, notamment pour passer d'une idée en 2D à un boîtier fonctionnel en 3D, ce qui est une compétence clé pour tout projet IoT embarqué nécessitant un boîtier spécifique.

Tests

Plusieurs tests ont été réalisés afin de valider le bon fonctionnement du système du nichoir connecté.

Test du capteur PIR

Le capteur PIR a été vérifié pour détecter correctement la présence d'oiseaux dans le nichoir. Des ajustements ont été effectués sur le potentiomètre de sensibilité pour limiter les déclenchements intempestifs dus aux mouvements extérieurs ou aux variations thermiques. Le réglage de la temporisation (Pot Time) a été testé pour s'assurer que la LED IR et la capture de l'image disposent du temps nécessaire avant que le signal ne repasse à l'état "bas".

Transmission des images via MQTT

Le protocole MQTT a été testé en envoyant plusieurs images vers le serveur Raspberry Pi. La reconstruction des images fragmentées a été vérifiée et validée. Des tests de robustesse réseau ont également été réalisés, avec interruption volontaire de la connexion, pour observer la reprise et la fiabilité des transmissions.

Autonomie énergétique

L'autonomie du système a été estimée en fonction de son fonctionnement actuel, avec un ESP32 partiellement alimenté pour le PIR. Une comparaison théorique a été faite avec le mode veille complet du vrai M5Stack Sleep, qui éteint totalement l'ESP32. Ces tests ont permis de mesurer l'impact des phases de veille, de la consommation de la LED et du PIR sur la durée de fonctionnement de la batterie.

Les résultats ont montré que le système remplit son rôle, tout en soulignant les limites liées à la gestion de l'énergie et aux interruptions réseau.

Limites

Malgré les résultats encourageants obtenus lors des tests, certaines limites du système ont été identifiées :

Réveil depuis le mode veille du PIR

Le principal point faible concerne le réveil de l'ESP32 depuis le vrai mode sleep profond fourni par le M5Stack. Le capteur PIR n'arrive pas à déclencher le réveil lorsque le

microcontrôleur est totalement éteint. Cela limite l'autonomie maximale du système, car il faut rester dans un mode veille partiel pour garantir la détection de mouvement.

Transmission réseau dépendante de la stabilité Wi-Fi

La fiabilité des envois d'images via MQTT dépend fortement de la qualité du réseau Wi-Fi. En cas de coupure ou de faible signal, certaines images peuvent être retardées ou nécessiter une retransmission.

Autonomie réelle vs autonomie théorique

L'autonomie estimée avec le mode veille partiel est inférieure à celle qui serait obtenue avec le vrai M5Stack sleep. Cela implique qu'une batterie plus grande ou une optimisation logicielle serait nécessaire pour une autonomie prolongée sur plusieurs mois sans intervention.

Exposition aux éléments

Même si le boîtier reste étanche dans sa position finale, il est soumis aux variations de température et d'humidité. Des tests supplémentaires seraient nécessaires pour valider un usage long terme en extérieur

Conclusion

Le projet nichoir connecté a atteint son objectif principal : créer un système autonome capable de détecter la présence d'oiseaux, de capturer des images et de les transmettre vers un serveur distant.

Les tests ont montré que le capteur PIR fonctionne correctement, que la synchronisation avec la LED et la caméra est efficace, et que la transmission via MQTT permet de récupérer les images et les métadonnées dans de bonnes conditions. Le boîtier 3D conçu protège l'ensemble des composants tout en assurant une aération suffisante et un câblage sécurisé.

Le système est opérationnel et démontre la faisabilité d'un nichoir connecté à faible coût et à autonomie optimisée. Malgré certaines contraintes techniques, notamment

liées au mode sleep complet de l'ESP32, le projet a permis d'acquérir des compétences concrètes en IoT, impression 3D, électronique et réseau.

En résumé, l'objectif de conception et de démonstration d'un nichoir intelligent fonctionnel est atteint, et le système peut servir de base pour des améliorations futures et des expérimentations supplémentaires.

Perspectives

Le nichoir connecté offre déjà un prototype fonctionnel, mais plusieurs axes d'amélioration peuvent être envisagés pour augmenter l'autonomie, la portée et la robustesse du système:

Intégration d'un réseau LoRa Mesh

L'utilisation de la technologie LoRa en mesh permettrait d'étendre la portée du nichoir et de créer un réseau de plusieurs dispositifs communicants. Grâce à cette approche, plusieurs nichoirs pourraient partager leur position et leurs données en temps réel, même dans des zones où le Wi-Fi n'est pas disponible. Cette méthode est inspirée de projets précédents réalisés avec un T-Beam LoRa GPS, démontrant la faisabilité d'un réseau décentralisé et autonome.

Alimentation solaire

L'ajout d'un petit panneau solaire permettrait de recharger la batterie du nichoir et d'assurer une autonomie prolongée, réduisant ainsi la nécessité d'interventions humaines. Combiné à une gestion optimisée de l'énergie et à des modes de veille efficaces, le système pourrait fonctionner plusieurs mois voire années de manière autonome.

Optimisation du mode veille

L'étude d'un deep sleep plus performant, éventuellement compatible avec le réveil par le PIR ou par un timer, pourrait améliorer considérablement l'autonomie tout en conservant une détection fiable des mouvements.

Extension des fonctionnalités

D'autres capteurs environnementaux (température, humidité, luminosité) pourraient être intégrés pour enrichir les données collectées et permettre une analyse plus complète du comportement des oiseaux et des conditions du nichoir.

Scalabilité et modularité

Grâce à l'architecture modulaire du boîtier et de l'électronique, il est possible de reproduire le système pour plusieurs nichoirs et de centraliser les données sur un serveur unique ou un cloud, facilitant les expérimentations à plus grande échelle.

Ces perspectives montrent que le projet peut évoluer vers un véritable système IoT de suivi écologique, combinant autonomie énergétique, communication longue portée et modularité, tout en conservant un coût accessible et une approche pédagogique.